An Exact Characterization of the Two-shot Deterministic Objects Solving Two-process Consensus

Minh Tung Nguyen
Telecom SudParis, Inria Saclay
Institut Polytechnique de Paris
Palaiseau, France

Pierre Sutra
Telecom SudParis, Inria Saclay
Institut Polytechnique de Paris
Palaiseau, France

Abstract

A key question in distributed computing is determining the synchronization power of a shared object. This power is commonly measured using consensus, a distributed problem in which processes agree on a common input value. An object has consensus number n when n is the largest number for which we may solve consensus with copies of this object and registers. The seminal work of Herlihy and Ruppert [14] provides an exact characterization of the consensus number for deterministic one-shot objects (that can be accessed by each process at most once). This paper extends that study to deterministic two-shot objects (that can be accessed by each process at most twice) in a two-process system. We introduce three disjoint classes of two-shot objects: The first class is similar to one-shot objects in the sense that the first operation call gives enough information to solve consensus. Objects in the second class do not provide any useful information after the first call to one of the two processes. The last class contains objects for which calling the object twice is always necessary. In this class, the second operation to call is chosen adaptively, which may lead to using different operations in different schedules. For instance, the second operation used in a solo run might differ from the one called when processes interleave. We show that these three classes provide an exact characterization of the two-shot deterministic objects able to solve two-process consensus. To establish this, we first prove that any two-shot deterministic object solving two-process consensus must belong to one of the three classes. Then, we present matching consensus algorithms, one for each class.

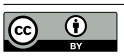
CCS Concepts

• **Theory of computation → Complexity classes**; **Distributed algorithms**; **Distributed computing models**.

Keywords

shared memory; consensus hierarchy; valency argument

ACM Reference Format:
Minh Tung Nguyen and Pierre Sutra. 2025. An Exact Characterization of the Two-shot Deterministic Objects Solving Two-process Consensus. In *ACM Symposium on Principles of Distributed Computing (PODC '25), June 16–20, 2025, Huatulco, Mexico.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3732772.3733534



This work is licensed under a Creative Commons Attribution 4.0 International License.
PODC '25, Huatulco, Mexico
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1885-4/2025/06
https://doi.org/10.1145/3732772.3733534

1 Introduction

Context. To solve complex tasks, modern machines rely on parallel architectures. Programming these architectures requires synchronizing accesses to shared resources. For that purpose, the hardware offers synchronization primitives such as test-and-set, compare-and-swap, and LL/SC. These primitives allow reading and modifying memory atomically [10]. They are used to build a variety of coordination mechanisms (such as locks and barriers) for programmers.

Since synchronization is essential, a key question is what capabilities hardware should provide. An abstraction to answer this concern is consensus, which permits processes to agree on a common input value. In his seminal work [11], Herlihy shows that consensus, together with read/write registers, can implement any object. This allows one to measure the synchronization power of an object as the maximum number of processes that solve consensus using objects of that type and registers. The measure is called the consensus number. It permits classifying objects (possibly, a specific subset of them) into a hierarchy [16]. This hierarchy is *robust* when it is not possible to use multiple objects of lower consensus number to implement a higher consensus number object.

Motivation. Many papers investigate the synchronization power of objects by studying their consensus numbers. Talmage and Welch [22] show a generic proof to find lower and upper bounds on the consensus number of various abstract data types. Peterson, Bazzi and Neiger [19] present a sufficient condition for an object to be able to solve n-process consensus. A recent work [9] studies the consensus number of a cryptocurrency. Herlihy and Ruppert [14] provide an exact characterization of the consensus power of one-shot objects using a topological approach. Ruppert [21] proves a necessary and sufficient condition to solve n-process consensus for read-modify-write and readable objects. He also uses these characterizations to show that the consensus hierarchy is robust when restricted to such objects.

All these results provide insights about the landscape of consensus numbers. Yet, many objects have not been characterized to date. Filling this gap would help to make progress on a more fundamental question: does the consensus number provide a robust hierarchy to compare general deterministic shared objects? This central question in distributed computing is still presently open [3, 7, 16].

Contributions. In this paper, we extend the work of Herlihy and Ruppert [14] to two-shot objects. More precisely, we study the deterministic two-shot objects that are able to solve two-process consensus. This problem is the smallest non-trivial problem that differs from the question posed by one-shot objects. It is interesting because a process can access each shared object more than once. For a one-shot object, the first call must provide enough information to

solve consensus. However, with two-shot objects, the first operation might reveal insufficient information and the process then needs another call to decide. This second call can depend on the current schedule and the interleavings of the previous calls.

Leveraging this observation, we introduce three disjoint classes: The first class is the class of one-shot objects that can be used together with registers to solve 2-process consensus. Objects in the second class do not provide enough information after a single call to one of the two processes. The last class contains objects for which calling the object twice is always necessary. In this class, the next operation to call is chosen adaptively, which may lead to using different operations in different schedules. For instance, the operation used in a solo run might differ from the one called when processes interleave.

We show that these three classes provide an exact characterization of the two-shot deterministic objects able to solve two-process consensus. To establish this, we first prove that any two-shot deterministic object solving two-process consensus must belong to one of the three classes. Then, we present matching consensus algorithms, one for each class.

Outline. This paper is organized as follows: Section 2 surveys related work. Section 3 presents the system model and the problem we are interested in. Section 4 introduces three disjoint classes of deterministic two-shot objects. For each class, Section 5 explains how to solve two-process consensus. Section 6 establishes that any two-shot deterministic object that solves two-process consensus must belong to one of these classes. Section 7 concludes the paper.

2 Related Work

Herlihy [11] proposes the notion of consensus number. He also establishes the universality of consensus in asynchronous, wait-free systems. This means that any deterministic, linearizable object can be implemented in a wait-free manner using n-consensus objects together with atomic read/write registers. This work also proves the irreducibility of consensus for any n: it is impossible to solve consensus among $2n$ processes by combining algorithms that solve consensus among $2m$ processes (with $m < n$). Jayanti and Toueg [17] extend this result for all n instead of $2n$. Furthermore, they also prove that finding the consensus number of an object is, in general, not decidable.

Since consensus is universal and irreducible, it is interesting to characterize objects based on their power to solve consensus. Jayanti [16] proposes a hierarchy (h^r_m) to classify all objects based on their consensus numbers. A hierarchy is *robust* when no collection of objects with consensus number n can be used to solve consensus in an $(n+1)$-process distributed system. Several papers try to prove the robustness of the h^r_m wait-free hierarchy [5, 19].

Herlihy and Ruppert [14] show a topology-based condition for a one-shot linearizable object to solve n-process consensus in a wait-free manner. This condition is based on the notion of k-solo-connectivity. Roughly speaking, this condition requires that, for any state s, for any k operations, any two permutations σ and σ' of the operations are connected by indistinguishability. This means that there exists a path $\sigma_1 \dots \sigma_{m\geq 2}$ with $\sigma_1 = \sigma$ and $\sigma_m = \sigma'$ such that for any $1 \leq i < m$, the response of some operation in σ_i and σ_{i+1} is the same. In the present paper, we show the existence of two-shot objects which are 2-solo connected when restricted to be one-shot, yet that solve consensus among two processes. Differently from [14], our characterization is based on a valency argument [8].

Ruppert [21] provides a necessary and sufficient condition for an object to be able to solve consensus in a wait-free manner. Two classes of objects are characterized: read-modify-write (RMW) and readable. A readable object has an operation that reads the object's state without changing it. A RMW object only has read-modify-write operations (such an operation reads the object's state, may modify it, then returns the previous state). The characterization requires the ability to assign one operation per process such that *(i)* processes can be split in two teams, say blue and red, and *(ii)* each process may distinguish runs that start with a blue operation from the ones that start with a red operation. Not all objects fall in these two classes (for instance, a queue limited to $enqueue()/dequeue()$ is neither readable, nor RMW). In contrast, our characterization is limited to two-process systems but it covers all deterministic two-shot objects.

Afek, Ellen, and Gafni [1] show that for all $i \geq 2$ there exists an infinite hierarchy of deterministic objects, each weaker than the next, which all have consensus number i. This disproves the common2 conjecture which suggested that objects in the common2 class [2] were strong enough to implement all objects with consensus number two (in arbitrarily large systems). There exist also an infinite hierarchy of deterministic objects having consensus number one [6]. These results show that the consensus number is an imperfect characterization of the synchronization power. Ellen, Gelashvili, Shavit, and Zhu [7] consider another measure based on the space usage to solve obstruction-free consensus.

Other works investigate the consensus numbers of shared objects, or classes of them. Bazzi, Neiger, and Peterson [4] prove that registers are not needed for solving consensus, because almost any type can be used to implement a one-shot shared bit. Borowsky, Gafni, and Afek [5] study deterministic linearizable objects in an attempt to prove that the hierarchy is robust. They also present an algorithm to initialize a (multi-shot) object to any reachable state. Kleinberg and Mullainathan [18] study the consensus power of objects in a limited-resources model, where the number of shared objects is restricted. More recently, Talmage and Welch [22] define several common classes of objects and establish both lower and upper bounds on their consensus numbers.

3 Preliminaries

System model. We follow the model in [16]. There are two deterministic processes, p_0 and p_1. Each process p_i has identifier i. During a run, a process may crash. In that case, the process is *faulty*; otherwise, it is *correct*. Processes have access to a private memory to do local computation. They communicate using shared objects. Shared objects are linearizable [15] and are specified by a sequential deterministic data type. The data type defines (1) a set of states $\mathcal{S}$; (2) a set of operations C that can be performed on the object; (3) a set of their response values $\mathcal{V}$; and (4) a transition function $\tau : \mathcal{S} \times C \to \mathcal{S} \times \mathcal{V}$. In the following, we assume a special null symbol ($\perp$) not in $\mathcal{V}$. When applying an operation f from a state s, we write $\tau_s(s, f)$ and $\tau_v(s, f)$ for the state and response of the operation, respectively. Without loss of generality, we consider that

every operation in C is applicable in every state in $\mathcal{S}$. For some sequence of operations $\sigma = c_1 \dots c_{k\geq 2}$, we write $s.\sigma$ to denote the state reached after applying the operations of σ in order. The response of $c_i \in \sigma$ from s is defined as $\tau_v(s.\sigma', c_i)$, where σ' is the prefix of σ preceding c_i.

Accessing a shared object can be restricted. For instance, a single-producer multiple-consumer queue permits a unique process to invoke *enqueue*(), while any process may call *dequeue*(). To account for this, a shared object is fully defined by a pair (T, m) where T is the sequential data type of the object, and m is the *process-allowance map* that associates to each process the operations of T the process may invoke. The pair (T, m) is called the concurrent data type of the object. Given some object O, we use $O.T$ to denote the sequential data type of O (and $O.m$ to denote its process-allowance map). Registers are the standard data type that permits processes to read from and write to the shared memory. We do not restrict the values stored in them.

Our notion of implementation is the one defined by Jayanti [16]. This definition permits the initial state of a shared object to be chosen freely. In detail, an implementation of object O in initial state s from objects $O_1, O_2, \dots$ in initial states $s_1, s_2, \dots$ respectively, for processes p_0 and p_1, consists of two procedures (one for each of the two processes) that specify how each process should perform the operations on O using operations on $O_1, O_2, \dots$. We say that O is the implemented object and that $O_1, O_2, \dots$ are the base objects. An implementation of object O (in initial state s) is linearizable when the history of O in every run is linearizable (and legal for the initial state s). It is wait-free if in all infinite runs and for every process p_i the following is true: if p_i is correct and has no incomplete operations on the base objects of O, then p_i has no incomplete operation on O.

The problem. In this paper, we focus on objects with limited number of uses. More precisely, an object is *one-shot* when each process calls it at most once in a run. Similarly, it is *two-shot* if it can be used at most twice per process. Consensus is a one-shot shared object defined over a set of values V. It supports the operation *propose*(v), for $v \in V$, that returns a value in V. A process that calls *propose*(v) *proposes* value v. When the call returns some value u, the process *decides* u. In every run, consensus ensures that: *(Validity)* If a process decides u then u was proposed previously. *(Agreement)* If a process decides u while another decides v then $u = v$. Hereafter, we consider binary consensus where only 0 or 1 are proposed. This variation is equivalent to the multivalued version [20]. An object solves the two-shot two-process consensus problem when there exists a wait-free and linearizable implementation of consensus with only objects of that type and registers.

4 The Characterization

In a run, a process observes the content of the shared objects over time. When it sees the same contents in two different runs, these runs are indistinguishable. However, consensus demands processes to distinguish certain closely similar runs. This section formalizes the notion of distinguishability for two-shot objects. The three classes having the power to solve two-process consensus are then presented.

4.1 Notion of Distinguishability

In what follows, O is a shared object of type (T, m) and s is a state of O. We assume that the operations $f, f' \in m(p_0)$ and $g, g' \in m(p_1)$ are executable by p_0 and p_1, respectively.

Our characterization is based on the notion of distinguishability, that is the ability of a process to see the difference between two interleavings, one starting with f and the other starting with g. In detail, consider the pair of sequences ff' and gg'. An *interleaving* of these sequences is a permutation σ of the operations in the two sequences such that in σ, f precedes f' and g precedes g'. By a slight abuse of terminology, we also refer to any prefix of an interleaving as an interleaving. For instance, the interleavings in which process p_0 runs solo first are ff', $ff'g$, and $ff'gg'$. Two interleavings σ and σ' are *distinguishable to* p_0 in state s when the response from s of either f or f' is different in σ and σ'. Similarly, σ and σ' are distinguishable to p_1 in s when the responses from s of g or g' are different in them. When s is clear from the context, we omit it. If two interleavings are not distinguishable, we sometimes say that they are *indistinguishable*.

Using the above notions, we introduce three disjoint classes of two-shot objects. This paper establishes that the two-shot objects with consensus power two are exactly in these three classes.

4.2 Three Classes

4.2.1 Rationale. Our characterization comes from the standard valency argument used to prove that consensus is impossible in various systems [8, 12]. In detail, consider a deterministic algorithm that solves consensus. Applying a valency argument, there exists a critical configuration (i.e., a global state) for this algorithm. In this configuration, the two processes are poised to access the same object, say O. Let f and g be the operations of O that processes p_0 and p_1 are poised to call, respectively. From this configuration, the runs that start with f decide a different value than the ones that start with g. Based on whether p_0 and p_1 know which of f or g comes first, using the responses of their calls, O belongs to one of the following three classes:

- If fg and gf are distinguishable to both processes, then O is in class $\mathscr{C}_1$.
- If fg and gf are distinguishable to exactly one of p_0 and p_1, then O is in class $\mathscr{C}_{1.5}$.
- Otherwise, fg and gf are indistinguishable to both processes, and O belongs to class $\mathscr{C}_2$.

Hereafter, we further detail the distinguishability power each class provides to solve consensus.

4.2.2 Class $\mathscr{C}_1$. The definition of class $\mathscr{C}_1$ is given below. To reach an agreement, a process uses the response of its first call to the object and figures out which of the two operations comes first. The full details are in Section 5.1.

Definition. *An object O is in class $\mathscr{C}_1$ when for some state s there exist operations $f \in O.m(p_0)$ and $g \in O.m(p_1)$ such that:*

(1) fg and gf are distinguishable to both processes.

4.2.3 Class $\mathscr{C}_{1.5}$. For objects in class $\mathscr{C}_{1.5}$, a single process sees the difference between fg and gf. Consequently, to solve consensus,

we require that objects in this class provide additional distinguishability power. The definition of class $\mathscr{C}_{1.5}$ appears below. It has five properties whose rationale will be explained shortly. In Section 5.2, we prove that consensus can be solved using class $\mathscr{C}_{1.5}$ and registers.

Definition. *Object O is in class $\mathscr{C}_{1.5}$ if it does not belong to class $\mathscr{C}_1$ and for some state s, there exist operations $f, f' \in O.m(p_0)$ and $g, g' \in O.m(p_1)$, satisfying:*

(1) fg and gf are distinguishable to p_0,
(2) fg and gf are indistinguishable to p_1,
(3) $fgg'f'$ and $gfg'f'$ are distinguishable to both processes,
(4) $gg'ff'$ and $fgg'f'$ are distinguishable to both processes, and
(5) either
(a) $gg'f$ and f are distinguishable to p_0, or
(b) $gg'ff'$ and $fgf'g'$ are distinguishable to both processes.

Class $\mathscr{C}_{1.5}$ also contains objects for which the roles of p_0 and p_1 are interchanged.

We now provide some intuition about properties *(1-5)*. To this end, we consider an object in class $\mathscr{C}_{1.5}$ for which p_0 is the process that distinguishes between fg and gf.

A base observation is that processes can read/write registers to discriminate between the solo runs of the two processes. In detail, consider that each process has a flag. Right after the first operation, the process raises its flag. Then, after the second operation, it checks the flag of the other process. If the flag is not raised, then the interleaving of the current run is not one in which the other process runs solo first. This mechanism permits processes to distinguish between $ff'gg'$ and $gg'ff'$.

A second observation is that, because fg and gf are indistinguishable to p_1, p_1 must access the object a second time. Now, because p_1 accesses the object a second time, it potentially impacts p_0. For instance, assume that p_1 accesses the object a second time using operation g'. If $gg'f$ and f are indistinguishable to p_0, then p_0 is also required to access the object a second time. Let f' be the second operation p_0 calls in runs that start with f or ggf'.

We require that p_0 can distinguish $gg'ff'$ from both $fgg'f'$ and $fgf'g'$. In the definition, this corresponds to the properties *(4)* and *(5b)*, respectively. With this, the object now permits p_0 to distinguish the interleavings that start from f from the ones that start with g.

A third observation is that p_0 can pass information to p_1 using the shared memory. For instance, p_0 may write the result of f to a register before calling f'. This permits p_1 to know if f or g comes first in interleavings where f' happens before g'. For the remaining interleavings, the object needs additional distinguishability power to enable p_1 to solve consensus.

In detail, process p_1 has to distinguish the following three pairs of interleavings: $fgg'f'$ from $gfg'f'$, $fgg'f'$ from $gg'ff'$, and $gg'ff'$ from $fgf'g'$. This corresponds to respectively properties *(3)*, *(4)* and *(5b)*. Notice that this last pair is not needed if *(5a)* $gg'f$ and f are distinguishable to p_0. Indeed, if this holds, then p_0 can inform p_1 that f was applied first in $fgf'g'$.

4.2.4 Class $\mathscr{C}_2$. The last class contains objects for which two calls are always necessary. The definition appears below and we explain its rationale hereafter. How to solve consensus with objects in class $\mathscr{C}_2$ is described in Section 5.3.

Definition. *Object O is in class $\mathscr{C}_2$ when it is not in class $\mathscr{C}_1$ or class $\mathscr{C}_{1.5}$ and for some state s, there exist $f \in O.m(p_0)$, $g \in O.m(p_1)$ and a sequence of operations $c_0, \ldots, c_n \in C$, where $n \geq 1$ and odd, with $c_i \in O.m(p_0)$ if i is even and $c_i \in O.m(p_1)$ if i is odd, satisfying the following properties:*

(1) fg and gf are indistinguishable to both processes,
(2) for all $0 \leq i < n$,
- fgc_ic_{i+1} and gfc_ic_{i+1} are distinguishable to both processes,
- fgc_ic_{i+1} and $gfc_{i+1}c_i$ are distinguishable to both processes,
- $fgc_{i+1}c_i$ and gfc_ic_{i+1} are distinguishable to both processes,
- $fgc_{i+1}c_i$ and $gfc_{i+1}c_i$ are distinguishable to both processes.
(3) fc_0gc_1 and gfc_0c_1 are distinguishable to both processes,
(4) either fc_0g and g are distinguishable to p_1, or fc_0gc_1 and gfc_1c_0 are distinguishable to both processes.
(5) gc_nfc_{n-1} and fgc_nc_{n-1} are distinguishable to both processes,
(6) either gc_nf and f are distinguishable to p_0, or gc_nfc_{n-1} and $fgc_{n-1}c_n$ are distinguishable to both processes.

With objects in class $\mathscr{C}_2$, the two processes cannot distinguish fg from gf. This is stated in property *(1)*. Hence, both processes must make a second call. A key observation is that, before making such a call, processes can access other objects. This permits them to gain information regarding what the other process does. As a consequence, the second operation can be chosen adaptively. Conversely, if a process cannot distinguish a sequence of such steps from another, then it must call the exact same second operation.

In detail, the intuition is as follows: Starting from the critical configuration, consider runs of the following form: After f and g get applied, the two processes take steps until they both choose a second operation, then, once their choices are settled, they make a second call. In run $\mathcal{E}_0$, process p_0 takes steps solo first followed by p_1. This means that $\mathcal{E}_0 = fg\sigma c_0c_1$, where σ are steps on objects other than O, and c_0 and c_1 are the operations chosen by p_0 and p_1, respectively. The opposite happens in run $\mathcal{E}_n$. Process p_1 runs solo followed by p_0, and they choose operations c_{n-1} and c_n, respectively. In Section 6, we prove that there exists a sequence of runs $\mathcal{E}_0 \ldots \mathcal{E}_n$, such that any two consecutive runs cannot be distinguished by at least one process. Each run $\mathcal{E}_i$ is of the form $fg\sigma' c_ic_{i+1}$, with $c_i \in O.m(p_0)$ if i is even and $c_i \in O.m(p_1)$ if i is odd. The operations chosen in these runs are the ones in the preamble of the definition of class $\mathscr{C}_2$.

Now, suppose that no process runs solo first before calling the second operation of object O. The following runs are possible from the critical configuration: $fg\sigma c_ic_{i+1}$ as well as $fg\sigma c_{i+1}c_i$, $gf\sigma c_ic_{i+1}$ and $gf\sigma c_{i+1}c_i$. Thus, the object must provide enough power to distinguish which of f and g comes first. This is property *(2)* in the definition of class $\mathscr{C}_2$.

If a processes runs solo first, the distinguishability power of the object should deal with additional interleavings. They are specified in the last four properties *(3-6)*. In detail, properties *(3)* and *(4)* are about c_0 and c_1. They correspond to properties *(4)* and *(5)* in class $\mathscr{C}_{1.5}$ with the roles of p_0 and p_1 interchanged. Similarly, properties *(5)* and *(6)* are about c_{n-1} and c_n, and they correspond to the last two properties of the class $\mathscr{C}_{1.5}$.

4.2.5 Examples. To illustrate the three classes, consider a FIFO queue with the following operations: *enqueue*(x) inserts x, *dequeue*() fetches the first element in the queue, and *read*() that returns the

state of the queue. When $m(p_0) = m(p_1)$ consists of all three operations, p_0 and p_1 have access to the whole interface; we call this object a readable queue (RQ). Now, if p_0 is unrestricted but p_1 may only call *enqueue*() and *read*(), the object is called an asymmetric queue (AQ). Last, when both processes are forbidden to use *dequeue*(), the object is a weak queue (WQ). In light of these definitions:

- RQ is in $\mathscr{C}_1$: The queue initially contains one element, say 0, and $f = g = dequeue()$.
- AQ is in $\mathscr{C}_{1.5}$: The queue is initialized as above, with $f = dequeue()$, $f' = read()$, $g = enqueue(1)$, and $g' = read()$.
- WQ is in $\mathscr{C}_2$: The queue is empty. We set $f = enqueue(0)$, $g = enqueue(1)$, and $c_0 = c_1 = read()$.

The rationale for this classification is as follows: First, each of the three queue objects (RQ, AQ, and WQ) satisfies the definition of the corresponding class. If $g = read()$, g does not change the state of the object. Thus, fg and gf result in the same state and the two interleavings are indistinguishable to p_0. If $g = enqueue(i)$, g does not return a response and fg and gf are indistinguishable to p_1. Therefore, AQ and WQ are not in $\mathscr{C}_1$ since p_1 does not have a suitable operation for g. If fg and gf are distinguishable to p_0 then $f = read()$, $g = enqueue(1)$ and the queue is initially empty. However, since the state of WQ is the same after fg and gf, $fgg'f'$ and $gfg'f'$ are indistinguishable to p_1. Thus, WQ is not in $\mathscr{C}_{1.5}$.

5 Solving Consensus

In this section, we show that consensus can be solved using a two-shot object O in any of the three classes. All the solutions follow the same pattern: Each process p_i writes down its consensus proposal to a single-writer register Val$[i]$. Then, p_0 calls f and p_1 calls g, where f and g are operations that witness O is in the class. Next, the process executes a sequence of read and write operations on the shared registers, and (possibly) make a second call. Using the response values of the two operations and the contents of the shared registers, the process then figures out whose operation was applied first to O. The corresponding proposal is the consensus decision.

5.1 Class $\mathscr{C}_1$

This is a known result from the literature [14, 21]. We briefly recall it here. Consider an object O of class $\mathscr{C}_1$. Since fg and gf are distinguishable to both processes, from the response of the call, p_i knows if f or g is applied first. If its operation comes first, p_i decides immediately its proposal. Otherwise, it decides the content of the register Val$[1-i]$.

5.2 Class $\mathscr{C}_{1.5}$

Alg. 1 gives the code for process p_0, while the code for p_1 appears in Alg. 2. The class of objects $\mathscr{C}_{1.5}$ is asymmetric. This explains why the two processes execute different algorithms.

5.2.1 Overview. Processes need to distinguish interleavings starting with f from the ones starting with g. The distinction between $ff'gg'$ and $gg'ff'$ is made using one register per process: Between the first and second access to O, a process writes its first response from O to its own register Ret$[i]$ then reads the other's. If the value read is null, this means that the other process did not run solo.

Algorithm 1 Consensus with a $\mathscr{C}_{1.5}$ object, code for p_0.

Shared memory:
- Object O of type (T, m); initially in state s.
- Registers: Val[2], Ret[2]; each register is initialized to $\perp$.

Local memory:
- Registers: v[2][2]; each register is initialized to $\perp$.

```
1:  Val[0] ← v_0
2:  v[0][0] ← O.f
3:  Ret[0] ← v[0][0]
4:  v[1][0] ← Ret[1]
5:  v[0][1] ← O.f'
6:  if v[1][0] = ⊥
7:      ▷ gg'ff' will never be here
8:      ▷ ff'gg' will always be here
9:      if v[0][0] = τ_v(s, f)
10:         return Val[0]
11:     return Val[1]
12: if v[0][0] ≠ τ_v(s, f)
13:     ▷ gf will always be here
14:     return Val[1]
15: ▷ fg or gg'ff' left
16: if v[0][0] = τ_v(s.gg', f) ∧ v[0][1] = τ_v(s.gg'f, f')
17:     ▷ gg'ff'
18:     return Val[1]
19: return Val[0]
```

Algorithm 2 Consensus with a $\mathscr{C}_{1.5}$ object, code for p_1.

```
20: Val[1] ← v_1
21: v[1][0] ← O.g
22: Ret[1] ← v[1][0]
23: v[0][0] ← Ret[0]
24: v[1][1] ← O.g'
25: if v[0][0] ≠ ⊥
26:     ▷ gg'ff' will never be here
27:     ▷ ff'gg' will always be here
28:     if v[0][0] = τ_v(s, f)
29:         return Val[0]
30:     return Val[1]
31: ▷ no ff'gg' after this point.
32: v[0][0] ← Ret[0]
33: if v[0][0] ≠ ⊥
34:     ▷ everything ending with g' is here
35:     if v[0][0] ≠ τ_v(s, f)
36:         ▷ gf will always be here
37:         return Val[1]
38:     if v[0][0] = τ_v(s.gg', f) ∧ v[1][1] = τ_v(s.g, g')
39:         ▷ starting with gg'
40:         return Val[1]
41:     ▷ starting with f
42:     return Val[0]
43: ▷ fgg'f', gg'ff', and gfg'f' are left
44: ▷ by assump., fgg'f' dist. from gg'ff' and gfg'f'
45: if v[1][0] = τ_v(s.f, g) ∧ v[1][1] = τ_v(s.fg, g')
46:     ▷ fgg'f'
47:     return Val[0]
48: return Val[1]
```

For the other interleavings, as fg and gf are distinguishable to p_0, p_0 can distinguish interleavings that start with f from the ones

that start with gf. However, the case $gg'ff'$ remains. To deal with it, p_0 leverages the fourth and fifth properties of the class $\mathscr{C}_{1.5}$.

For the interleavings $ff'gg'$, $fgf'g'$ and $gff'g'$, p_1 learns the first response p_0 obtained from O by reading Ret[0]. Since fg and gf are distinguishable to p_0, p_1 also learns which of f and g was applied first. For the other three interleavings, $gg'ff'$, $gfg'f'$ and $fgg'f'$, the third, fourth and fifth properties enable p_1 to know which of these occurred.

5.2.2 Details. The solution combines the ideas above in an efficient manner. It uses two arrays of registers shared among the processes: Val and Ret. Val stores proposals. Ret contains the response value of the first operation executed by each process on object O. Object O is initially in state $s \in \mathcal{S}$. State s and the operations $f, f' \in O.m(p_0)$, and $g, g' \in O.m(p_1)$ satisfy the conditions given in Section 4.2.3. Each process also uses a local two-dimensional array, v. The entry v[i][j] stores the response of the j-th operation on O made by process p_i. Initially, each register contains the null value ($\perp$).

The solution works as follows: First, each process shares its proposal. Then, it accesses object O twice. The response values of these calls is stored locally in v and the response of the first call is stored in the shared variable Ret. The process uses the content of v and the content of Ret[$1 - i$] to determine which interleaving of operations happened, and it decides accordingly.

The decision procedure is in lines 6 to 19 for process p_0 (Alg. 1), and in lines 25 to 48 for p_1 (Alg. 2). The goal of the procedure is to figure out which of f or g comes first. For this, it uses the content stored in variable v. As the procedure progresses, it eliminates the interleavings that cannot happen (see the comments in green). When the remaining interleavings all start with the same operation, the procedure returns the proposal of the corresponding process.

The first "if" block, in lines 6 to 11 for p_0 and lines 25 to 30 for p_1, eliminates $gg'ff'$. By assumption, process p_0 distinguishes between the interleavings that start with f and gf. Thus, it may decide once it enters the block. Process p_1 also enters this block when it knows the response of f (v[0][0] $\neq \perp$ in line 25). It follows the same steps as p_0 to decide.

The second block for process p_0 is in lines 12 to 14. It determines if f was not applied first—in which case p_0 can safely decide Val[1]. Otherwise, if the test in line 12 is false, it eliminates all the interleavings starting with gf. Note that at this stage, if f and $gg'f$ are indistinguishable to p_0, then $gg'ff'$ cannot be eliminated yet. This happens in the last block. Process p_0 decides Val[1] in case it observes the interleaving $gg'ff'$, and Val[0] otherwise.

Regarding process p_1, the first block (lines 25 to 30) decides if $ff'gg'$ happens. After this block, p_1 reads again Ret[0]. If its content is non-null, p_1 knows the return value of f. It then follows a logic similar to p_0 (lines 12 to 19) to decide which of f or g comes first. Process p_0 uses the return value of f' in a single place, at line 16, to distinguish $gg'ff'$ from $fgf'g'$ and $fgg'f'$. Instead, p_1 makes this distinction with the return value of g', using the fourth and fifth properties of class $\mathscr{C}_{1.5}$. If Ret[0] contains $\perp$, p_1 is left with $fgg'f'$, $gg'ff'$, and $gfg'f'$. Thanks to the third and fourth properties of $\mathscr{C}_{1.5}$ objects, $fgg'f'$ is distinguishable from the two others.

5.2.3 Correctness. In what follows, we establish that the proposed approach is correct. First, we prove that processes return the content of the same dedicated register in a run.

Lemma 1. *A process returns Val[0] if f occurs before g during the run, and Val[1] otherwise.*

Proof. For starters, we show that p_0 knows which operation f or g was applied first. Let us examine the first "if" block in lines 6 to 11 for p_0. In case it happens, we know that p_0 does not see the result of g. Hence, p_0 reads Ret[1] before p_1 writes to it. Therefore, f, which happens before p_0 reads Ret[1], happens before g', which happens after p_1 writes to Ret[1]. It follows that the interleaving $gg'ff'$ is impossible. As a consequence, the first operation f of p_0 is applied either on s (the initial state of O), or on the state $s.g$. Thanks to the assumption that fg and gf are distinguishable to p_0, p_0 may decide which of the two happened using the response value stored in v[0][0].

Alternatively, consider that p_0 observes v[1][0] $\neq \perp$ at line 6. This means that the read at line 4 happens after line 22, which implies that f' happens after g. Hence, if the test on line 6 is false, then the interleaving $ff'gg'$ has been eliminated.

If the response of f is not the response of f from state s, p_0 did not take the first step on object O. Thus, when the block in lines 12 to 14 is executed, process p_0 can safely decide Val[1]. Otherwise, assume that the response of f is the same as the response of f from state s at line 12. Since f and gf are distinguishable to p_0, we can eliminate the interleavings that start with gf. Therefore, we are left with at most 3 interleavings, namely $fgf'g'$, $fgg'f'$, and $gg'ff'$, before line 16 is performed. The last block in lines 16 to 19 takes care of distinguishing $gg'ff'$ from the other two. It relies on the fact that O is a $\mathscr{C}_{1.5}$ object. In detail, $gg'ff'$ and $fgg'f'$ are distinguishable to p_0 by the fourth property, and $gg'ff'$ and $fgf'g'$ are distinguishable to p_0 by the last property. This explains the test at line 16.

Using a similar reasoning, we can show that p_1 knows which of the two operations is applied to object O first. The interleaving $ff'gg'$ is caught in lines 25 to 30 of Alg. 2. At line 32, process p_1 reads again the content of Ret[0] to retrieve the response of operation f. If it does not read $\perp$, we can rely on the same arguments as the ones employed for p_0 to prove that the decision logic is sound. (This corresponds to lines 12 to 19 in Alg. 1.) Alternatively, consider that v[0][0] $= \perp$. It implies that line 32 happens before line 3, that is g' happens before f'. Thus, only the interleavings $gg'ff'$, $gfg'f'$, and $fgg'f'$ are left. At line 45, process p_1 checks if the response values of g and g' are the ones in the interleaving $fgg'f'$. If this happen to be the case, it return Val[0]; otherwise Val[1]. Thanks to the third and fourth properties of object O, which discriminates $fgg'f'$ from the other two interleavings, this computation is correct. □

Theorem 1. *Processes p_0 and p_1 solve consensus by executing respectively Algs. 1 and 2.*

Proof. In Algs. 1 and 2, a process takes a finite number of steps before deciding. Hence, the two algorithms are wait-free. Lemma 1 establishes that if a process decides, it decides the value proposed by the process whose operation on O was applied first. Before executing such an operation, the process must have written its proposal to a dedicated register in the array Val (line 1 in Alg. 1 and line 20 in Alg. 2). Because the run is linearizable, this value is recorded in Val when a process decides. Thus, every decision is a proposed value. From what precedes, and by applying Lemma 1 again, the two processes reach an agreement during every run. □

5.3 Class $\mathscr{C}_2$

We now turn our attention to the last class identified in Section 4. Alg. 3 presents an implementation of consensus using one object O in class $\mathscr{C}_2$. This algorithm assumes a state $s \in \mathcal{S}$, two operations $f \in O.m(p_0)$ and $g \in O.m(p_1)$, as well as a sequence of operations $c_0, \ldots, c_{n\geq 1} \in C$. They satisfy the characterization given in Section 4.2.4 for this class of objects. The lemma below establishes that, without loss of generality, we may assume that $n = 3^k$ with $k > 0$.

Lemma 2. *Assume an object O such that for some $s \in \mathcal{S}, f \in O.m(p_0), g \in O.m(p_1)$, and $c_0, \ldots, c_{n\geq 1} \in C$, O is $\mathscr{C}_2$. Then, there exist $c'_0, \ldots, c'_m \in C$, with $m = 3^k$ and $k > 0$, for which O is $\mathscr{C}_2$.*

Proof. Consider the largest $k > 0$ such that $n = 3^{k-1} + a$, with $a \geq 0$. Let $c'_i = c_i$ for all $0 \leq i \leq n$. In case $a = 0$, we are done. Otherwise, we can extend inductively the sequence $c_0, \ldots, c_{n\geq 1}$ by adding $c'_{n+1} = c_{n-1}$ and $c'_{n+2} = c_n$. Since n is odd, so is $n+2$. We have $c'_{n+1} \in O.m(p_0)$ and $c'_{n+2} \in O.m(p_1)$. Because c_{n-1} and c_n satisfy the third property of $\mathscr{C}_2$, c_n and c_{n+1} satisfy this property as well. Similarly, c_{n+1} and c_{n+2} also satisfy this property. By assumption c_{n-1} and c_n satisfy the last property, which is also true for c_{n+1} and c_{n+2}. Extending the sequence as explained previously is repeated until $n = 3^k$. □

5.3.1 Choosing the Next Operation. The first steps of Alg. 3 follow a similar pattern as the previous solutions. Initially, each process shares its proposal using register Val. Then, it executes the first operation, recording its response in Ret. (lines 1 to 3).

The key idea of Alg. 3 is the protocol to choose the second operation, in lines 5 to 17. This protocol works similarly to a standard solution to the 3^k-approximate agreement problem in a two-process system [13].

In detail, the loop at lines 8 to 15 executes $k > 0$ iterations, with $n = 3^k$ (thanks to Lemma 2). In this loop, each process chooses an operation in the sequence $c_0, \ldots, c_{n\geq 3}$ to apply next to object O. The restriction $k > 0$ implies $n \geq 3$, which helps the algorithm separate two runs where p_0 has a solo run first (p_0 chooses c_0) and p_1 has a solo run first (p_1 chooses $c_{n\neq 1}$).

Variable *cnt* tracks the number of iterations made so far in the loop. In each iteration, process p_i ponders which second operation to apply to object O. Variable op stores its current choice. When an iteration starts, p_i stores its choice in the entry Round$[i][cnt]$ of the shared array Round. If it observes that other_op was chosen by p_{1-i} at the current iteration, p_i chooses op + other_op × 2 (line 12). Otherwise, p_i chooses operation $3 \times op$ (line 14).

Fig. 1 illustrates such an approach when $k = 2$. Solid nodes (●) represent choices for p_0, and hollow ones (○) choices for p_1. Arrows model transitions between iterations. In each iteration (top to bottom), p_i either sees p_{1-i}'s choice at that round (in red), or not (in blue). Arrows in bold describe a potential run of the loop. In this run, in the first iteration, p_0 sees p_1 but p_1 does not see p_0. Then, the two processes see each other in the second iteration.

Let op_i be the value of variable op at process i at the end of a run. We can prove the following lemma.

Lemma 3. *In every run of Alg. 3, $|op_0 - op_1| = 1$.*

Algorithm 3 Consensus with a $\mathscr{C}_2$ object, code at p_i.

Shared memory:
- Object O of type T; initially in state s.
- Registers: Val[2], Ret[2], Round[2][k], Next_op[2]; each register is initialized to $\perp$.

Local memory:
- Register: v[2][2]; each register is initialized to $\perp$.
- Register: other_op; each register is initialized to $\perp$.

```
1:  Val[i] ← v_i
2:  v[i][0] ← (i = 0) ? O.f : O.g
3:  Ret[i] ← v[i][0]
4:
5:  ▷ start protocol to choose the next operation
6:  cnt ← 0
7:  op ← i
8:  while cnt < k do
9:      Round[i][cnt] ← op
10:     other_op ← Round[1 − i][cnt]
11:     if other_op ≠ ⊥
12:         op ← op + other_op × 2
13:     else
14:         op ← op × 3
15:     cnt + +
16: Next_op[i] ← op
17: ▷ end protocol to choose the next operation
18:
19: v[i][1] ← O.c_op
20: v[1 − i][0] ← Ret[1 − i]
21:
22: ▷ f c_0 g c_1
23: if op = 0 ∧ v[0][0] = τ_v(s, f) ∧ v[0][1] = τ_v(s.f, c_0)
        ∧ (v[1][0] = ⊥ ∨ v[1][0] = τ_v(s.f c_0, g))
24:     return Val[0]
25: if op = 1 ∧ v[1][0] = τ_v(s.f c_0, g)
        ∧ v[1][1] = τ_v(s.f c_0 g, c_1) ∧ Next_op[0] = 0
26:     return Val[0]
27: ▷ g c_{3^k} f c_{3^k − 1}
28: if op = 3^k ∧ v[1][0] = τ_v(s, g) ∧ v[1][1] = τ_v(s.g, c_{3^k})
        ∧ (v[0][0] = ⊥ ∨ v[0][0] = τ_v(s.g c_{3^k}, f))
29:     return Val[1]
30: if op = 3^k − 1 ∧ v[0][0] = τ_v(s.g c_{3^k}, f)
        ∧ v[1][1] = τ_v(s.g c_{3^k}.f, c_{3^k − 1}) ∧ Next_op[1] = 3^k
31:     return Val[1]
32: ▷ start with either fg or gf
33: if Next_op[1 − i] = ⊥
34:     if v[i][1] = τ_v(s.fg, c_op)
35:         return Val[0]
36:     return Val[1]
37: if v[i][1] = τ_v(s.fg c_{Next_op[1−i]}, c_op) ∨ v[i][1] = τ_v(s.fg, c_op)
38:     return Val[0]
39: return Val[1]
```

Proof. We split the run into successive asynchronous rounds. Round r consists of all the steps taken by the processes when variable *cnt* equals r. The proof is by induction on the round number.

Initially, the property holds because op equals i at process p_i. Now, consider some round r. In round r, a process p_i either sees the process p_{1-i} writing or not. In the former case, op_i is set to

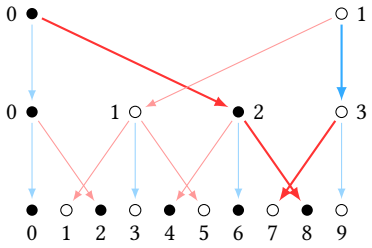


Figure 1: Choice of the second operation in Alg. 3 ($k = 2$).

$op_i + 2 \times op_{1-i}$ (line 12), and in the later case to $3 \times op_i$ (line 14). Suppose that, at round r, process p_i performs line 9 before p_{1-i} does. When process p_{1-i} performs line 10, it will read the value that p_i wrote. Thus, at least one process can read the value that the other wrote. This leads to three distinct situations to consider (they are illustrated in Fig. 1).

In what follows, we write op'_i and op''_i the value of op_i before and after it get updated at round r, respectively. First, assume that p_0 sees p_1 but p_1 does not see p_0. Then, $op''_0 = op'_0 + 2 \times op'_1$ and $op''_1 = 3 \times op'_1$ in the next round. By induction, $|op''_0 - op''_1| = |op'_0 + 2 \times op'_1 - 3 \times op'_1| = |op'_0 - op'_1| = 1$, and the property holds at round $r + 1$. The case where p_1 sees p_0 but p_0 does not see p_1 is symmetrical to the previous one. If the two processes see each other, the difference at round $r + 1$ is $|op'_0 + 2 \times op'_1 - (op'_1 + 2 \times op'_1)| = |op'_1 - op'_0| = 1$ and the property holds. Thus, $|op_0 - op_1| = 1$ after each iteration. □

A direct consequence of Lemma 3 and the definition of a $\mathscr{C}_2$ object that the interleaved accesses of the processes to object O are distinguishable to both processes. It is used in the decision procedure that we cover later on.

Corollary 1. *In any run of Alg. 3, the following pairs of interleavings are distinguishable to both processes:* $(fgc_{op_0}c_{op_1}, gfc_{op_0}c_{op_1})$, $(fgc_{op_0}c_{op_1}, gfc_{op_1}c_{op_0})$, $(fgc_{op_1}c_{op_0}, gfc_{op_0}c_{op_1})$, *and* $(fgc_{op_1}c_{op_0}, gfc_{op_1}c_{op_0})$.

Proof. From Alg. 3, variable op_i is in $[0, k]$, op_0 is even, and op_1 odd. By Lemma 3, it is true that $|op_0 - op_1| = 1$. The result is then a direct consequence of the second property listed in the definition of class $\mathscr{C}_2$ (see Section 4.2.4). □

5.3.2 Making the Decision. Once the second operation is chosen, it is applied to object O and the response is stored in the local array v (line 19). Then, the process attempts to read the response of the first operation applied by the other process (line 20). This terminates the protocol part of the consensus algorithm. The decision procedure appears in lines 22 to 39. Its logic is explained below.

For starters, suppose that $fc_{op_0}gc_{op_1}$ occurs. In that case, $op = 0$ at process p_0 and $op = 1$ at process p_1. At process p_0, the condition in line 23 is true. It is also false for any interleaving that starts with g. Similarly, the condition in line 26 takes care of $fc_{op_0}gc_{op_1}$ for process p_1.

The interleaving $gc_{3^k}fc_{3^k-1}$, where p_1 runs solo first, is sorted out in lines 28 to 31. The is symmetrical to the previous case.

The rest of the decision procedure deals with interleavings that start with either fg or gf (lines 33 to 39).

5.3.3 Correctness. The decision procedure permits knowing which of p_0 or p_1 accesses object O first, ensuring agreement. Below, we state such a result then establishes that Alg. 3 is correct.

Lemma 4. *In Alg. 3, a process returns the input of p_0 if f happens before g and the input of p_1 otherwise.*

Proof. Lines 23 to 26 capture interleavings in which p_0 alone accesses object O first, that is $fc_{op_0}gc_{op_1}$. In what follows, we prove that the conditions of the two "if" blocks are indeed always true for fc_0gc_1, possibly true for other interleavings that start with f (for which processes should also return Val[0]), and always false for any interleaving starting with g. We examine in turn process p_0 then p_1 for which $op = 0$ and $op = 1$, respectively.

(Case $op = 0$.)

- **$\mathbf{fc_0gc_1}$**: The test in line 23 checks that the responses of operations match the interleaving fc_0gc_1. More precisely, it checks that the return values of f and c_0 are the same as in fc_0gc_1, and that the return value of g is either not available to p_0, or as in fc_0gc_1, Hence, the condition is necessarily true when fc_0gc_1 happens.
- **$\mathbf{gfc_0c_1}$**: Since O is in class $\mathscr{C}_2$, the third property says that fc_0gc_1 and gfc_0c_1 are distinguishable to p_0. Thus, if the interleaving is gfc_0c_1, at line 23, either $\mathrm{v}[0][0] = \tau_v(s, f)$ or $\mathrm{v}[0][1] = \tau_v(s.f, c_0)$ is false.
- **$\mathbf{gfc_1c_0}$**: By contradiction, suppose gfc_1c_0 occurs yet p_0 returns at line 24. First, notice that $\mathrm{v}[1][0] \neq \perp$ at p_0. Indeed, p_0 reads $\mathrm{v}[1][0]$ (line 20) after it applies c_0 to O (line 19). This happens after p_1 executes c_1 (line 19), which is after when p_1 writes the response of g to Ret[1] (line 3). According to line 23, $\mathrm{v}[1][0] = \tau_v(s.fc_0, g)$. By applying the fourth property of $\mathscr{C}_2$, fc_0g and g are distinguishable to p_1 ($\tau_v(s, g) \neq \tau_v(s.fc_0, g)$), or fc_0gc_1 and gfc_1c_0 are distinguishable to p_0 (i.e., $\tau_v(s.f, c_0) \neq \tau_v(s.gfc_1, c_0)$, since $\tau_v(s, f) = \tau_v(s.g, f)$ by the first property of class $\mathscr{C}_2$). From $\mathrm{v}[1][0] = \tau_v(s.fc_0, g)$, fc_0g and g are indistinguishable to p_1. Hence $\tau_v(s.f, c_0) \neq \tau_v(s.gfc_1, c_0)$ and the condition in line 23 fails; contradiction.
- **$\mathbf{gc_1fc_0}$**: This is impossible because $op = n$ would hold at p_1.

(Case $op = 1$.) Applying Lemma 3, the second operation of p_0 is either c_0 or c_2. Since line 25 checks $\mathrm{Next_op}[0] = 0$, it is c_0. In what follows, we verify that the condition in line 25 is necessarily true for fc_0gc_1, and false for interleavings starting with g.

- **$\mathbf{fc_0gc_1}$**: At line 25, the test verifies that the response values of operations g and c_1 match the interleaving fc_0gc_1. Moreover, since p_0 writes 0 to $\mathrm{Next_op}[0]$ at line 16, before it applies c_0, p_1 should have $\mathrm{Next_op}[0] = 0$ at line 16. Hence, the test at line 25 passes.
- **$\mathbf{gfc_0c_1}$**: By the third property of $\mathscr{C}_2$, fc_0gc_1 and gfc_0c_1 are distinguishable to both processes. Thus either $\tau_v(s.fc_0, g) \neq \tau_v(s, g)$ or $\tau_v(s.fc_0g, c_1) \neq \tau_v(s.gfc_0, c_1)$. This implies that in the interleaving gfc_0c_1, either $\mathrm{v}[1][0] = \tau_v(s.fc_0, g)$ or $\mathrm{v}[1][1] = \tau_v(s.fc_0g, c_1)$ is false.
- **$\mathbf{gfc_1c_0}$**: Another property of $\mathscr{C}_2$ is that either fc_0gc_1 and gfc_1c_0 are distinguishable, or $\tau_v(s, g) \neq \tau_v(s.fc_0, g)$. Thus, either $\tau_v(s, g) \neq \tau_v(s.fc_0, g)$ or $\tau_v(s.gf, c_1) \neq \tau_v(s.fc_0g, c_1)$. Hence, in the interleaving gfc_1c_0, either $\mathrm{v}[1][0] = \tau_v(s.fc_0, g)$ or $\mathrm{v}[1][1] = \tau_v(s.fc_0g, c_1)$ is false.

From what precedes, a process correctly decides the input of p_0 (stored in Val[0]) in the first two "if" blocks. The logic which takes care of $gc_{3^k} fc_{3^k-1}$ (in lines 28 to 31) is proved symmetrically.

In the remaining cases, we are left with interleavings that start with fg or gf (lines 33 to 39). How the decision is made in such a situation is detailed below.

First, assume that p_i reads $\perp$ from Next_op$[1-i]$. Process p_i applies c_{op_i} before reading Next_op$[1-i]$. Hence, its second operation on object O happens before $c_{op_{1-i}}$. Therefore, there are two possible interleavings, either $fgc_{op_i}c_{op_{1-i}}$ or $gfc_{op_i}c_{op_{1-i}}$. By Corollary 1, $fgc_{op_i}c_{op_{1-i}}$ and $gfc_{op_i}c_{op_{1-i}}$ are distinguishable. Because object O is $\mathscr{C}_2$, fg and gf are indistinguishable to both processes. It follows that $\tau_v(s.fg, c_{op_i}) \neq \tau_v(s.gf, c_{op_i})$. Thus, checking the response of the second operation at line 34 provides a sound decision.

In lines 37 to 39, process p_i retrieves the second operation of p_{1-i} from register Next_op$[1-i]$. By Lemma 3, it must be either c_{i-1} or c_{i+1}. Process p_i then uses the properties listed in Corollary 1 to decide based on the response value of its second operation. □

Theorem 2. *Alg. 3 implements consensus.*

Proof. Thanks to Lemma 4, Alg. 3 satisfies both validity and agreement. Because there are only a finite amount of steps before returning in any branches, this algorithm is wait-free. □

6 Necessary Conditions for Consensus

Suppose algorithm $\mathcal{A}$ solves two-process binary consensus using two-shot shared objects of type (T, m) and atomic registers. Furthermore, assume that $\mathcal{A}$ uses the *minimal* number of shared objects of any such algorithm. Below, we prove that objects of type (T, m) necessarily belong to one of the three classes.

Notation. We use a valency argument [8, 12]. For this, we consider the runs of $\mathcal{A}$. In a run, the states of the processes and the shared memory at some point in time are called a *configuration*. Configurations are connected to form a directed graph. There is an edge e labeled with p_i from configuration C to configuration D when the next operation of p_i in C leads to D. If c is an operation called in configuration C, then we denote by $C.c$ the configuration reached after applying it. By extension, this operator extends to a sequence of operations $\sigma = c_1 \ldots c_k$, and $C.\sigma$ is the configuration reached after applying them in order. Consider some configuration C reachable from the initial configuration of $\mathcal{A}$ by applying some sequence σ. A value $v \in \{0, 1\}$ is a decision value of C if some process already decided v in configuration C (that is, *propose*() at that process returns v in C). Let V be the set of decision values of configurations reachable from C. If $|V| = 1$, C is said to be *univalent*. When this happens, C is 0-valent or 1-valent according to the corresponding decision value. The configuration is *bivalent* when $|V| = 2$. When C is bivalent and all configurations reachable from it are univalent, C is *critical*.

Outline of the proof. As in standard valency arguments, one can show that there exists a critical configuration, C_0, for algorithm $\mathcal{A}$ in which processes are poised to access the same object O of type (T, m). This is necessary for C_0 to be critical. We denote by s the state of O in configuration C_0. The next operation called by p_0 is $f \in O.m(p_0)$, and $g \in O.m(p_1)$ is the next operation of p_1. We write $C_f = C_0.f$ and $C_g = C_0.g$ to denote the configurations reached after applying f and g, respectively. Since C_0 is critical, C_f and C_g have distinct valencies. As in [8], we apply an indistinguishability argument: Let D and D' be two configurations reachable from C_0. Suppose a process p_i performs the same sequence σ of operations to attain D and D' and these operations have the same responses. If p_i uses object O twice in σ and all the objects besides O are in identical states in D and D', then the two configurations have the same valency. With this reasoning, we show that O is necessarily in one of the three classes.

The proof is as follows: Lemma 5 is a technical lemma. Proposition 1 considers the case when O is in $\mathscr{C}_{1.5}$. Proposition 2 shows that if O is neither in $\mathscr{C}_1$ nor in $\mathscr{C}_{1.5}$, then it is in $\mathscr{C}_2$.

Lemma 5. *Let $\mathcal{E}_1 = fg\sigma_{p_1}g'\sigma_{p_0}f'$ be some execution starting from C_0 in which p_0 performs f, then p_1 executes the operations $g\sigma_{p_1}$ until it performs a second operation g' on O, after which p_0 executes σ_{p_0} then calls its second operation f' on O. If g and fg are indistinguishable to p_1, then*

- *$gg'ff'$ and $fgg'f'$ are distinguishable to both processes, and*
- *either $gg'f$ and f are distinguishable to p_0, or $gg'ff'$ and $fgf'g'$ are distinguishable to both processes.*

Proof. We first show that $gg'ff'$ and $fgg'f'$ are distinguishable to both processes. Consider each process in turn. *(Case p_1.)* As g and fg are indistinguishable to p_1, if p_1 starts a solo run from $C_0.g$, it executes the same operations $\sigma_{p_1}g'$ as in $\mathcal{E}_1$. Let $\mathcal{E}_2 = g\sigma_{p_1}g'$ be this execution starting from C_0. Since fg and g have different valencies, the responses of g' in $\mathcal{E}_1$ and $\mathcal{E}_2$ must be different. It follows that $gg'ff'$ and $fgg'f'$ are distinguishable to p_1. *(Case p_0.)* Consider the extension of $\mathcal{E}_2$ where, after p_1 performs g', p_0 runs solo. If f and $gg'f$ are distinguishable to p_0, then $fgg'f'$ and $gg'ff'$ are distinguishable to p_0. Otherwise, p_0 gets the same response for f as in $\mathcal{E}_1$ and therefore, it applies the same sequence of operations σ_{p_0} after $g\sigma_{p_1}g'f$. Let $\mathcal{E}_3 = g\sigma_{p_1}g'f\sigma_{p_0}f'$ be this execution from C_0. After $\mathcal{E}_3$, the objects other than O are in the same state as after $\mathcal{E}_1$. Since $\mathcal{E}_1$ and $\mathcal{E}_3$ lead to configurations with different valencies, the response of f' is different in $\mathcal{E}_3$. It follows that $gg'ff'$ and $fgg'f'$ are distinguishable to p_0.

Next, we show that the second property holds. If f and $gg'f$ are distinguishable to p_0, then we are done. Otherwise, consider the execution $fg\sigma_{p_1}$ from C_0. If p_0 runs solo until its next operation on O, these steps of p_0 are exactly the ones in $\mathcal{E}_1$. After these, p_0 calls operation f' then process p_1 executes operation g'. Let $\mathcal{E}_4 = fg\sigma_{p_1}\sigma_{p_0}f'g'$ be this execution. Applying a similar reasoning as above, this time on $\mathcal{E}_3 = g\sigma_{p_1}g'f\sigma_{p_0}f'$ and $\mathcal{E}_4$, we can show that $gg'ff'$ and $fgf'g'$ are distinguishable to the two processes. □

Proposition 1. *Suppose O is not in $\mathscr{C}_1$. If fg and gf are distinguishable to p_0 but indistinguishable to p_1 then O is in $\mathscr{C}_{1.5}$.*

Proof. First, we establish that $fgg'f'$ and $gfg'f'$ are distinguishable to both processes (third property of the class $\mathscr{C}_{1.5}$). The two interleavings are distinguishable to p_0 because by assumption fg and gf are distinguishable to p_0. Define $\mathcal{E}_1$ as in Lemma 5. After the operations gf, suppose that p_1 runs solo until it executes a second operation on object O. By assumption, the response of g in $\mathcal{E}_1$ and in this execution is the same. Hence, p_1 executes the same sequence $\sigma_{p_1}g'$. Let $\mathcal{E}_2 = gf\sigma_{p_1}g'$ be the corresponding execution from C_0.

Because C_f and C_g have different valencies, the response of g' in $\mathcal{E}_1$ and $\mathcal{E}_2$ must be different. It follows that $fgg'f'$ and $gfg'f'$ are distinguishable to p_1, as required. Then, because g and fg are indistinguishable to p_1, we may apply Lemma 5. It follows that the remaining properties in the definition of $\mathscr{C}_{1.5}$ are also met. □

Proposition 2. *If fg and gf are indistinguishable to both processes, then O is in $\mathscr{C}_2$.*

Proof. By assumption, p_0 and p_1 cannot decide right away from either $C_0.fg$, or $C_0.gf$. Hence, they take additional steps in $\mathcal{A}$ from these configurations. To characterize such steps, consider the following undirected graph G: The nodes of G are the operations on object O. There is an edge $\{c, c'\}$ in G iff for some execution of $\mathcal{A}$ starting from $C_0.fg$, p_0 and p_1 call respectively c and c' on object O in this execution, and they stop after returning from these calls.

Suppose p_0 and p_1 run solo one after the other until they access O again. Let $c_0 \in O.m(p_0)$ and $c_1 \in O.m(p_1)$ be these two next operations. In other words, we have the execution $fg\sigma_{p_0}\sigma_{p_1}c_0c_1$. Conversely, suppose that p_1 runs first followed by p_0. Let f' and g' be the second operations on O performed by p_0 and p_1, respectively. That is, the execution is $fg\sigma'_{p_1}\sigma'_{p_0}g'f'$. By construction, $\{c_0, c_1\}$ and $\{f', g'\}$ are two edges in G. We show that there exists a path between c_1 and f'.

The proof is by contradiction. Assume that the two operations are in distinct connected components of G. Then, we can solve consensus using strictly fewer objects of type (T, m) than in $\mathcal{A}$.

Construction. *The algorithm uses the variables of $\mathcal{A}$ initialized in configuration $C_0.fg$, but it does not employ object O. (This initialization is possible in our system model—see Section 3). At the start of the algorithm, each process writes its proposal to a dedicated register. Then, it performs a sequence of accesses to the objects other than O as in algorithm $\mathcal{A}$ starting from $C_0.fg$. Right before it executes a second operation on object O, it halts. If it is poised to perform an operation in the connected component of c_0, the process decides the proposal of p_0. Otherwise, it decides the proposal of p_1.*

This solves consensus for the following reasons: *(Validity)* When p_0 decides the value of p_1, p_1 has already written to its dedicated register. Indeed, if this is not the case, necessarily p_0 runs solo first. Hence p_0 is about to execute c_0, and it must instead decide its own value. A similar argument applies to process p_1. *(Agreement)* Consider the two operations c and c' chosen by p_0 and p_1 in a run of the algorithm. Neither p_0 nor p_1 take steps after c and c'. By definition, $\{c, c'\}$ is an edge in G, that is they are in the same connected component. As a consequence, p_0 and p_1 always decide the same value. The algorithm uses strictly fewer objects than $\mathcal{A}$, which leads to a contradiction.

Thus there exists a path between c_1 and f' in G. This path is denoted by the sequence $c_1, \ldots, c_{n-1} = f'$, with $n \geq 2$ even. We extend this path with c_0 and $c_n = g'$ (From what precedes, $\{c_0, c_1\}$ and $\{f', g'\}$ are two edges in G). In case $c_0 = f'$ and $c_1 = g'$, we can have a shorter sequence c_0, c_1 instead. Observe that for any $0 \leq i \leq n$, $c_i \in O.m(p_0)$ if i is even, and $c_i \in O.m(p_1)$ if i is odd. The set of operations $c_0, \ldots, c_n$ satisfies the requirements at the beginning of the definition of the class $\mathscr{C}_2$ given in Section 4.2.4.

Next, assume that c_i and c_{i+1}, with $0 < i < n-1$ and i odd, are linked by an edge. (The case when i is even is shown similarly.) By definition of G, there exists an execution of $\mathcal{A}$ in which p_0 and p_1 call respectively c_i and c_{i+1} on object O. This execution starting from C_0 is of the form $fg\sigma_1c_i\sigma_2c_{i+1}$, where *(i)* σ_2 only contains operation of p_1, and *(ii)* σ_1 and σ_2 only contain operations on objects other than O. Since σ_2 contains only operations of p_1 on objects other than O, we can move it before c_i. We obtain the execution $\mathcal{E}_1 = fg\sigma_1\sigma_2c_ic_{i+1} = fg\sigma c_ic_{i+1}$ from C_0. Moreover, because fg and gf are indistinguishable to both processes, $\mathcal{E}_2 = gf\sigma c_ic_{i+1}$ is also a possible execution of $\mathcal{A}$. Since $\mathcal{E}_1$ and $\mathcal{E}_2$ have different valencies and the state of all the objects other than O are identical, the responses of c_i and c_{i+1} must be different in these two executions. Thus, fgc_ic_{i+1} and gfc_ic_{i+1} are distinguishable to both processes. Similarly, because c_i and c_{i+1} are operations by different processes, they can be executed in any order. This leads to $\mathcal{E}_3 = fg\sigma c_{i+1}c_i$ and $\mathcal{E}_4 = gf\sigma c_{i+1}c_i$. Applying the reasoning above to $\mathcal{E}_3$ and $\mathcal{E}_4$, $fgc_{i+1}c_i$ and $gf\sigma c_{i+1}c_i$ are distinguishable to both processes. This completes the second property listed in Section 4.2.4.

By definition of f' and g', $fg\sigma'_{p_1}\sigma'_{p_0}g'f'$ is possible from configuration C_0. Since σ'_{p_0} and g' are by different processes and on distinct objects, we may swap them, leading to $fg\sigma'_{p_1}g'\sigma'_{p_0}f'$. This execution is $\mathcal{E}_1$ as defined in Lemma 5. Thus, we may apply Lemma 5, which gives us the fifth and sixth properties of $\mathscr{C}_2$ in Section 4.2.4.

Similarly, consider $fg\sigma_{p_0}\sigma_{p_1}c_0c_1$. Since fg and gf are indistinguishable to both processes, the execution $gf\sigma_{p_0}\sigma_{p_1}c_0c_1$ is also possible from C_0. Applying the same argument as above, the execution $gf\sigma_{p_0}c_0\sigma_{p_1}c_1$ is also possible from C_0. We apply Lemma 5 to $gf\sigma_{p_0}c_0\sigma_{p_1}c_1$, with the role of p_0 and p_1 swapped. This gives us the fourth and fifth properties in the definition of the class $\mathscr{C}_2$. □

Theorem 3. *If some algorithm solves two-process consensus using objects of type (T, m) and registers, then (T, m) belongs to either $\mathscr{C}_1$, $\mathscr{C}_{1.5}$, or $\mathscr{C}_2$.*

Proof. The theorem follows from a case analysis and the results in Propositions 1 and 2. □

7 Conclusion

In this paper, we present three disjoint classes of two-shot objects, two of them being new, that permit solving two-process consensus. We prove that if a two-shot object can solve two-process consensus, then it is necessarily in one of these three classes. At core, our work extends the characterization given by Herlihy and Ruppert [14]. A key difference with [14], which solely covers the one-shot case, is that the second operation is not pre-determined by the initial state. Instead, it depends on the schedule of the processes during the run and the interleaving of their calls to all the shared objects. The robustness of the wait-free hierarchy remains a fundamental question in distributed computing. Our study addresses a specific subproblem within this broader issue, which we hope will inform future research.

Acknowledgments

We thank the PODC reviewers for their helpful suggestions and comments. In particular, we are deeply grateful to our shepherd, Faith Ellen, for her invaluable guidance and insightful feedback in finalizing the paper. This work was supported by CHIST-ERA (grant ANR-23-CHR4-0009).

References

[1] Yehuda Afek, Faith Ellen, and Eli Gafni. 2016. Deterministic Objects: Life Beyond Consensus. In *Proceedings of the 2016 ACM Symposium on Principles of Distributed Computing* (Chicago, Illinois, USA) *(PODC '16)*. Association for Computing Machinery, New York, NY, USA, 97–106. doi:10.1145/2933057.2933116

[2] Yehuda Afek, Eytan Weisberger, and Hanan Weisman. 1993. A completeness theorem for a class of synchronization objects. In *Proceedings of the Twelfth Annual ACM Symposium on Principles of Distributed Computing* (Ithaca, New York, USA) *(PODC '93)*. Association for Computing Machinery, New York, NY, USA, 159–170. doi:10.1145/164051.164071

[3] James Aspnes. 2023. Notes on Theory of Distributed Systems. arXiv:2001.04235 [cs.DC] https://arxiv.org/abs/2001.04235

[4] Rida A. Bazzi, Gil Neiger, and Gary L. Peterson. 1997. On the Use of Registers in Achieving Wait-Free Consensus. *Distributed Comput.* 10, 3 (1997), 117–127. doi:10.1007/S004460050029

[5] Elizabeth Borowsky, Eli Gafni, and Yehuda Afek. 1994. Consensus power makes (some) sense! (extended abstract). In *Proceedings of the Thirteenth Annual ACM Symposium on Principles of Distributed Computing* (Los Angeles, California, USA) *(PODC '94)*. Association for Computing Machinery, New York, NY, USA, 363–372. doi:10.1145/197917.198126

[6] Eli Daian, Giuliano Losa, Yehuda Afek, and Eli Gafni. 2018. A Wealth of Sub-Consensus Deterministic Objects. In *32nd International Symposium on Distributed Computing (DISC 2018) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 121)*, Ulrich Schmid and Josef Widder (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 17:1–17:17. doi:10.4230/LIPIcs.DISC.2018.17

[7] Faith Ellen, Rati Gelashvili, Nir Shavit, and Leqi Zhu. 2020. A complexity-based classification for multiprocessor synchronization. *Distributed Comput.* 33, 2 (2020), 125–144. doi:10.1007/S00446-019-00361-3

[8] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. 1985. Impossibility of Distributed Consensus with One Faulty Process. *J. ACM* 32, 2 (April 1985), 374–382. doi:10.1145/3149.214121

[9] Rachid Guerraoui, Petr Kuznetsov, Matteo Monti, Matej Pavlovic, and Dragos-Adrian Seredinschi. 2022. The consensus number of a cryptocurrency. *Distributed Comput.* 35, 1 (2022), 1–15. doi:10.1007/S00446-021-00399-2

[10] John L. Hennessy and David A. Patterson. 2011. *Computer Architecture, Fifth Edition: A Quantitative Approach* (5th ed.). Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.

[11] Maurice Herlihy. 1991. Wait-free Synchronization. *ACM Trans. Program. Lang. Syst.* 13, 1 (Jan. 1991), 124–149. doi:10.1145/114005.102808

[12] Maurice Herlihy. 2016. Asynchronous Consensus Impossibility. In *Encyclopedia of Algorithms*. 152–155. doi:10.1007/978-1-4939-2864-4_36

[13] Maurice Herlihy, Dmitry Kozlov, and Sergio Rajsbaum. 2014. Chapter 2 - Two-Process Systems. In *Distributed Computing Through Combinatorial Topology*, Maurice Herlihy, Dmitry Kozlov, and Sergio Rajsbaum (Eds.). Morgan Kaufmann, Boston, 21–39. doi:10.1016/B978-0-12-404578-1.00002-4

[14] M. Herlihy and E. Ruppert. 2000. On the existence of booster types. In *Proceedings 41st Annual Symposium on Foundations of Computer Science*. 653–663. doi:10.1109/SFCS.2000.892333

[15] Maurice Herlihy and Jeannette M. Wing. 1990. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Trans. Program. Lang. Syst.* 12, 3 (1990), 463–492. doi:10.1145/78969.78972

[16] Prasad Jayanti. 1997. Robust wait-free hierarchies. *J. ACM* 44, 4 (jul 1997), 592–614. doi:10.1145/263867.263888

[17] Prasad Jayanti and Sam Toueg. 1992. Some Results on the Impossibility, Universality, and Decidability of Consensus. In *Proceedings of the 6th International Workshop on Distributed Algorithms (WDAG '92)*. Springer-Verlag, Berlin, Heidelberg, 69–84.

[18] Jon M. Kleinberg and Sendhil Mullainathan. 1993. Resource Bounds and Combinations of Consensus Objects. In *Proceedings of the Twelth Annual ACM Symposium on Principles of Distributed Computing, Ithaca, New York, USA, August 15-18, 1993*, Jim Anderson and Sam Toueg (Eds.). ACM, 133–143. doi:10.1145/164051.164069

[19] Gary L. Peterson, Rida A. Bazzi, and Gil Neiger. 1994. A gap theorem for consensus types extended abstract. In *Proceedings of the Thirteenth Annual ACM Symposium on Principles of Distributed Computing* (Los Angeles, California, USA) *(PODC '94)*. Association for Computing Machinery, New York, NY, USA, 344–353. doi:10.1145/197917.198123

[20] Michel Raynal. 2013. *Concurrent Programming - Algorithms, Principles, and Foundations*. Springer. doi:10.1007/978-3-642-32027-9

[21] Eric Ruppert. 2000. Determining Consensus Numbers. *SIAM J. Comput.* 30, 4 (2000), 1156–1168. doi:10.1137/S0097539797329439

[22] Edward Talmage and Jennifer Welch. 2016. Generic Proofs of Consensus Numbers for Abstract Data Types. In *19th International Conference on Principles of Distributed Systems (OPODIS 2015) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 46)*, Emmanuelle Anceaume, Christian Cachin, and Maria Potop-Butucaru (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 32:1–32:16. doi:10.4230/LIPIcs.OPODIS.2015.32